

CuraGo — Complete System Documentation

Single source of truth for how the deployed system (www.curago.in) works. Last updated: 2026-07-09. Stack: Next.js 16 (App Router, JS), React 19, MongoDB (Mongoose 9), Vercel.

Status legend used throughout: -  **LIVE** — actively used in production -  **PAUSED** — built and working, intentionally switched off -  **DORMANT** — code exists and works but is not the active path -  **LEGACY** — superseded, kept for backward compat; do not build on it

Table of Contents

1. What CuraGo Is
 2. Domain Routing & Middleware
 3. Environment Variables
 4. Data Models
 5. Roles & Authentication
 6. Doctor Public Website (Subdomain Sites)
 7. Patient Booking Flow — OTP (LIVE)
 8. Patient Booking Flow — Payment (DORMANT)
 9. Priority Connect (LIVE, single-tenant)
 10. Doctor Admin Dashboard
 11. Platform Admin Dashboard
 12. Sub-User Dashboards (Clinic Manager, SEO, Reputation Manager)
 13. Contacts CRM, Messaging & Workflows
 14. Google My Business / Reputation Suite (PAUSED)
 15. Subscriptions, Promo Codes & AI Tokens
 16. Custom Domains
 17. Legacy Dr. Yuvaraj Site (LIVE on root domain)
 18. Analytics & Tracking
 19. Cron Jobs
 20. SEO — Current State
 21. Scripts & Maintenance Tools
 22. Complete API Route Reference
 23. Known Issues & Technical Debt
-

1. What CuraGo Is

A **multi-tenant SaaS platform for doctors**: each doctor gets a website at `{subdomain}.curago.in` (or a custom domain) built from a section-based page builder, with WhatsApp-OTP-verified appointment booking, Google Calendar integration, a contacts CRM with automated SMS/email drip workflows, and a (currently paused) Google My Business reputation suite.

The platform **evolved from a single-tenant website for Dr. Yuvaraj T** (surgical gastroenterologist, Mumbai). That original site still runs on the root domain (`/blog` , `/gbsi` , `/services` , `/digital-clinic` , `/myclinic` , `/priority-connect` , `/schedule-consultation`) and is **live production traffic** — see §17.

Production facts (confirmed): - Active booking flow = **OTP flow only** (no per-booking payment collected). - **One shared Google Calendar** (`team@curago.in`) receives all tenants' events — intentional. - **GMB suite is paused** (feature-flagged off). - Legacy Dr. Yuvaraj pages are live.

2. Domain Routing & Middleware

File: `middleware.js` (root). Runs on all paths except `/api/*` , `_next/*` , files with extensions.

Host	Behavior
<code>curago.in</code> , <code>www.curago.in</code>	Normal routes → marketing site + legacy Yuvaraj pages
<code>{doctor}.curago.in</code>	Rewrite → <code>/site/{doctor}{path}</code> , header <code>x-subdomain</code> set
<code>admin.curago.in</code>	Rewrite → <code>/platform-admin{path}</code>
Any other domain (custom domain)	Calls <code>GET /api/public/domain-lookup?domain=...</code> to find the owning doctor's subdomain, then rewrites → <code>/site/{subdomain}{path}</code> with <code>x-subdomain</code> + <code>x-custom-domain</code> headers
<code>*.vercel.app</code> , <code>localhost</code>	Treated as root domain (no subdomain handling)

Root domain constant: `NEXT_PUBLIC_ROOT_DOMAIN` (default `curago.in`). `www` is not treated as a subdomain.

⚠ Note: doctor subdomains only have a homepage route — `/site/[subdomain]/page.js` has no child routes, so `{doctor}.curago.in/anything` 404s (see §23).

3. Environment Variables

Variable	Purpose	Used in
<code>MONGODB_URI</code>	MongoDB Atlas connection (<code>yuvaraj-booking</code> DB)	<code>lib/mongodb.js</code> , <code>lib/db.js</code> , scripts
<code>JWT_SECRET</code> / <code>SESSION_SECRET</code>	JWT signing for all roles (both set to the same value)	<code>lib/doctorAuth.js</code> , <code>lib/auth.js</code> , <code>lib/platformAdminAuth.js</code> , sub-user auth routes
<code>NEXT_PUBLIC_ROOT_DOMAIN</code>	<code>curago.in</code>	middleware, emails, subdomain resolution
<code>NEXT_PUBLIC_APP_URL</code>	<code>https://curago.in</code>	metadata base, GMB redirect, review links
<code>GOOGLE_SERVICE_ACCOUNT_EMAIL</code>	Service account for Calendar	<code>lib/googleCalendar.js</code>
<code>GOOGLE_PRIVATE_KEY</code> / <code>GOOGLE_PRIVATE_KEY_BASE64</code>	Service account key (base64 variant recommended on Vercel)	<code>lib/googleCalendar.js</code>
<code>GOOGLE_CALENDAR_ID</code>	<code>team@curago.in</code> — the single shared calendar	<code>lib/googleCalendar.js</code>
<code>DOCTOR_EMAIL</code>	Impersonation subject for domain-wide delegation	<code>lib/googleCalendar.js</code>
<code>DOCTOR_NAME</code>	Legacy display name	legacy pages
<code>WYLT0_OTP_API_KEY</code>	Wylto WhatsApp API (OTP + review requests)	<code>send-otp</code> , <code>cron/review-requests</code>
<code>TWILIO_ACCOUNT_SID</code> / <code>TWILIO_AUTH_TOKEN</code> / <code>TWILIO_PHONE_NUMBER</code>	SMS (OTP backup, confirmations, CRM messaging)	<code>lib/twilio.js</code>

Variable	Purpose	Used in
RESEND_API_KEY , EMAIL_FROM	Transactional email	lib/email.js , lib/messaging.js
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Razorpay REST API (subscriptions, dormant booking payments, AI tokens)	lib/razorpaySubscription.js , verify-payment , ai-tokens/*
NEXT_PUBLIC_RAZORPAY_KEY_ID	Client-side Razorpay key	checkout pages
RAZORPAY_PAYMENT_BUTTON_ID	Default payment button	payment-mode booking
RAZORPAY_WEBHOOK_SECRET	HMAC verification of subscription webhook only	doctor/subscription/webhook
PLATFORM_ADMIN_EMAIL / PLATFORM_ADMIN_PASSWORD	The single platform-admin account (env-only, no DB)	lib/platformAdminAuth.js
ADMIN_USERNAME / ADMIN_PASSWORD	 Legacy admin login	lib/auth.js
BLOB_READ_WRITE_TOKEN	Vercel Blob image uploads	api/admin/upload-image
GOOGLE_GMB_CLIENT_ID / GOOGLE_GMB_CLIENT_SECRET / GOOGLE_GMB_REDIRECT_URI	 GMB OAuth	lib/gmb.js
VERCEL_API_TOKEN / VERCEL_PROJECT_ID	Custom-domain management via Vercel Domains API	lib/vercelDomains.js
CRON_SECRET	Bearer auth for cron endpoints	api/cron/*
ANTHROPIC_API_KEY	AI website/blog generation (currently unset — “Coming Soon”)	lib/aiGenerate.js
NEXT_PUBLIC_SAMPLE_VIDEO_URL	Demo video on Blob CDN	marketing

4. Data Models

35 Mongoose models in `models/` . `doctorId` is **optional** on booking/scheduling models for backward compat with legacy single-tenant data; uniqueness is enforced with sparse compound indexes.

Tenancy & identity

Model	Purpose	Key fields
Doctor	Tenant root	email/phone (unique), bcrypt password, subdomain (unique, reserved-list validated), customDomain , displayName, specialization, profileImage, bio, isEmailVerified (+ email OTP), isActive , whatsappNumber , wyltoWebhookId , referral codes, platformReferenceCode
ClinicManager	Doctor-scoped sub-user (contacts+workflows)	email, bcrypt password, doctorId
SEOUser	Doctor-scoped sub-user (website+blog editing)	same shape as ClinicManager
ReputationManager	Platform-scoped agency user	assignedDoctors[]
Subscription	One per doctor	plan (trial/monthly/premium), status, razorpaySubscriptionId, trial/period dates, ₹1000 default

Model	Purpose	Key fields
PromoCode	Premium unlock codes (e.g. CURAG050)	unlocksPremium, freeSmsCredited, maxUses, usedBy[]
ReferenceCode	Signup gate codes (e.g. CURAG02024)	maxUses, usedBy[] — overlaps conceptually with PromoCode
OTP	Patient booking OTPs	phone, otp, TTL index (5 min), 3 attempts max, embedded full bookingData

Website builder

Model	Purpose
BookingPage	The doctor's website. sections[] (18 types: header, hero_carousel, banner_image, benefits_list, doctor_profile, testimonials, faqs, faq_chatbot, location_map, disease_icons_scroll, custom_text, cta_button, booking_form, clinic_info, professional_fees, footer, whatsapp_sticky, book_now_sticky — each {type, order, visible, config}), theme (forest/ocean/sunset/royal/teal/coral), SEO meta (title/metaDescription/metaKeywords), paymentMode (no_payment = LIVE default / payment), consultationFee (1000), bookingFee (150), razorpayButtonId, status (draft/published/archived), views/bookings counters
BlogArticle	 Single-tenant clinical blog (hardcoded “Dr. Yuvaraj T”), rigid 6-section schema, no doctorId
ForumPost	 Public Q&A, no doctorId
Clinic	Doctor's clinic locations (address, timings, mapUrl, fees, isPrimary)

Scheduling & bookings

Model	Purpose
ConsultationMode	Per-doctor booking modes (e.g. online / in-clinic), unique {doctorId, name}
TimeSlot	Per-doctor 30-min slot definitions (“HH:MM”), unique {doctorId, time}
WeeklySchedule	Per doctor + mode + dayOfWeek: isEnabled + enabledSlots[]
DateOverride	Per-date block or custom slots
Booking	Patient booking: patient info, mode/modelId, date/time, status (pending_payment / confirmed / expired / cancelled), expiryTime, paymentId, eventId / meetLink / calendarEventUrl
Slot	 Legacy global slot model — superseded by TimeSlot
MeetingLink	Doctor’s saved meeting URLs — not used by booking flow (Meet links stored on Booking)

CRM & messaging

Model	Purpose
Contact	CRM contact (doctorId, phone/email, status string, tags, source: manual/import/booking/website)
ContactStatus	Custom pipeline stages per doctor (6 defaults seeded)

Model	Purpose
MessageTemplate	SMS/email templates with <code>{{name}}</code> , <code>{{reviewLink}}</code> , etc.
MessageLog	Every sent message (channel, status, Twilio SID / Resend id)
MessageQuota	Monthly per-doctor limits: 100 SMS / 500 email, atomic deduction
Workflow	Multi-step drip campaign: steps <code>{delayDays, delayHours, channel, templateId}</code> ; 5 defaults seeded
WorkflowExecution	Running instance per contact: <code>currentStepIndex</code> , <code>nextRunAt</code> , logs
AIToken	AI credit wallet per doctor (purchases via Razorpay, usage log)

GMB / reputation (🛑 paused)

Model	Purpose
GmbConnection	OAuth tokens (select:false), account/location ids, placeId, per-feature toggles
GmbPost	Scheduled/published GMB posts with retry + analytics
GmbReview	Synced Google reviews + <code>aiSuggestedReply</code> + reply state
GmbFaq	GMB Q&A
GmbInsight	Time-series metrics (search keywords, views, clicks, direction requests)
ReviewRequest	Post-visit review request: channel, <code>trackingId</code> (unique), status funnel (pending→sent→clicked→reviewed/intercepted), interceptor rating/feedback
ReviewRequestTemplate	Per-channel request templates, <code>delayHours</code> default 24

Analytics

Model	Purpose
SlotView	Funnel tracking: patient details captured on “View Slots” click; <code>convertedToBooking</code> / <code>bookingId</code> fields exist but are never set (future)

5. Roles & Authentication

All roles use **JWT in an httpOnly cookie**, all signed with the **same secret** (`JWT_SECRET` , falling back to `SESSION_SECRET`). Roles are distinguished only by the `role` claim in the token. Dashboards are guarded **client-side only** (layout `useEffect` → `fetch /auth/me` → `redirect`); there is no server middleware guard.

Role	Login page	Cookie	Store	Expiry	Scope
Doctor	<code>/login</code>	<code>doctor_token</code>	Doctor (bcrypt)	30d	Own tenant, full admin
Platform admin	<code>admin.curago.in/login</code>	<code>platform_admin_token</code>	env vars only , plaintext compare	7d	Everything
Clinic manager	<code>/clinic-manager/login</code>	<code>clinic_manager_token</code>	ClinicManager (bcrypt)	7d	One doctor: contacts + workflows
SEO user	<code>/seo/login</code>	<code>seo_token</code>	SEOUser (bcrypt)	7d	One doctor: website builder + blog

Role	Login page	Cookie	Store	Expiry	Scope
Reputation manager	/reputation-manager/login	rep_manager_token (note: not reputation_manager_token)	ReputationManager (bcrypt)	7d	Assigned doctors: contacts only
 Legacy admin	/api/admin/login	returns Bearer token in body	env ADMIN_USERNAME / ADMIN_PASSWORD	30d	legacy admin routes

Key libraries: - `lib/doctorAuth.js` — the linchpin. `getCurrentDoctor()` reads `doctor_token`, and if absent/invalid **falls back to `seo_token`**, then `clinic_manager_token`, resolving either to the *owning doctor's full document*. This is how sub-user dashboards reuse doctor pages/APIs unchanged — **sub-role restrictions exist only in the UI; the API grants full doctor access** (see §23). - `lib/platformAdminAuth.js` — env-credential login, `requirePlatformAdmin`. - `lib/auth.js` —  legacy `isAuthenticated` (accepts `doctor_token` cookie or Bearer). Some `/api/admin/*` routes still use this weaker check.

Doctor signup flow (`POST /api/auth/signup`): 1. Validates subdomain (regex + reserved list in `lib/doctorAuth.js`), optional `ReferenceCode`. 2. Creates Doctor (bcrypt), generates **email OTP** (10-min) → sent via Resend. 3. `POST /api/auth/verify-email` → sets `isEmailVerified`. Doctor must be verified + active to log in and for their site to render. 4. Trial `Subscription` auto-created (30 days). 5. Password reset: hashed token, 1-hour expiry (`forgot-password` / `reset-password`).

6. Doctor Public Website (Subdomain Sites)

Entry: `app/site/[subdomain]/page.js` (server component).  LIVE

Render pipeline: 1. Middleware rewrote `{doctor}.curago.in/` → `/site/{doctor}`. 2. Look up `Doctor` by subdomain — must be `isActive` and `isEmailVerified`, else `not-found.js` (“Site Not Found”). 3. Load the **oldest published** `BookingPage` for that doctor; increment `views`. 4. No page/sections → minimal fallback card (avatar, tel: link, “Powered by CuraGo”). 5. Otherwise: sections sorted by `order`, sticky buttons split out, each mapped to a component via an inline `switch` (`renderSection`). Components live in `components/booking-page/sections/`. 6. If doctor has `whatsappNumber` and no sticky configured, a default WhatsApp sticky button is auto-added.

Theming: `lib/themes.js` — 6 preset palettes; applied as `data-theme={id}` on the wrapper; `app/globals.css` `[data-theme="..."]` blocks override `--color-primary-*` CSS vars.

Metadata: `generateMetadata` uses `BookingPage` `title` / `metaDescription`; reads `bookingPage.ogImage` which **doesn't exist in the schema** → doctor sites never emit an OG image.

Second renderer: `components/booking-page/SectionRenderer.js` is a parallel implementation used by `/myclinic/[slug]` (legacy surface). The two can drift; the `SectionRenderer` omits `header`.

Public JSON twin: `GET /api/public/[subdomain]/site` returns the same doctor+page data as JSON (not used by the SSR page; likely legacy/external consumer).

7. Patient Booking Flow — OTP (LIVE)

 **This is the production booking path.** Trigger: `booking_form` section (`components/booking-page/sections/BookingFormSection.js`) with `paymentMode: 'no_payment'` (the default).

```

Patient on {doctor}.curago.in
|
├─ 1. GET /api/consultation-modes?doctorId=... → doctor's active ConsultationModes
├─ 2. Patient picks mode + date (7-day window; "today" blocked in UI → WhatsApp CTA)
├─ 3. GET /api/available-slots?date=...&modeId=...&doctorId=...
|   lib/slotManagerDB.getEffectiveSlotsForDate():
|   WeeklySchedule(mode, dayOfWeek).enabledSlots
|   - DateOverride blocks/custom slots
|   - past slots + 60-min buffer
|   - already-booked slots (confirmed OR pending_payment, across ALL modes)
|   (also auto-releases expired reservations first)
├─ 4. Patient fills name/age/gender/email/whatsapp → "View Slots" fires
|   POST /api/track-slot-view (SlotView analytics, non-blocking)
├─ 5. Submit → POST /api/send-otp
|   • doctorId resolved from Host subdomain
|   (⚠ fallback: FIRST active+verified doctor if unresolvable)
|   • 6-digit OTP + full bookingData stored in OTP collection
|   (5-min TTL index, max 3 attempts)
|   • OTP sent via Wylto WhatsApp API (template otp_template)
|   AND Twilio SMS backup – failures don't abort
├─ 6. Patient enters OTP → POST /api/verify-otp-and-book {phone, otp}
|   • OTP.verifyOTP → returns stored bookingData
|   • Re-check slot still free (409 if taken)
|   • Google Calendar event created (lib/googleCalendar.js)
|   - HARD REQUIREMENT: booking fails if Calendar API fails
|   - Google Meet link auto-generated when mode === 'online'
|   - event lands on the SHARED team@curago.in calendar (intentional)
|   • Booking saved: status 'confirmed', eventId, meetLink, calendarEventUrl
|   • Wylto webhook POST (bookingType 'no_payment') – WhatsApp automation
|   • Patient: Resend confirmation email + Twilio SMS
|   • Doctor: Resend notification email + Twilio SMS (whatsappNumber || phone)
|   (all notifications best-effort, try/catch)
└─ 7. Widget shows inline "Booking Confirmed" card (no redirect)

```

Google Calendar mechanics (lib/googleCalendar.js): JWT service-account auth with **domain-wide delegation**, impersonating DOCTOR_EMAIL (team@curago.in). Service accounts cannot email attendees (Google policy) — this is why confirmations go out via Resend/Twilio/Wylto instead of calendar invites (documented in CURRENT_SYSTEM_CAPABILITIES.md).

OTP providers: primary **Wylto** (https://server.wylto.com/api/v1/wa/send , Bearer WYLT0_OTP_API_KEY); backup **Twilio SMS**. In dev, the OTP is returned in the API response.

8. Patient Booking Flow — Payment (DORMANT)

🔒 Fully built, not the active path (no doctor uses paymentMode: 'payment' today).

```

Same steps 1-4 as OTP flow, then:
├─ POST /api/reserve-slot
│   └─ Booking created with status 'pending_payment', expiryTime = now + 10 min
│       (the reservation IS a Booking row; countdown shown in widget)
│   └─ reservationId + pendingBooking stored in sessionStorage
├─ Razorpay Payment Button injected (data-payment_button_id = page.razorpayButtonId,
│   default pL_S32iD93nAACoNH)
├─ Razorpay redirects → /payment-callback?razorpay_payment_id=...
├─ POST /api/verify-payment {razorpay_payment_id, reservationId}
│   └─ Real signature present → HMAC-SHA256 verify with RAZORPAY_KEY_SECRET
│   └─ Payment-Button flow (signature === "payment_button") → server fetches
│       GET api.razorpay.com/v1/payments/{id} and requires status captured/authorized
│   └─ Reservation expired → 410
│   └─ Calendar event created → confirmReservation flips Booking to 'confirmed'
│   └─ Same Wylto webhook + emails + SMS as OTP flow
└─ Redirect → /schedule-confirmation (renders sessionStorage data)

```

⚠️ **No Razorpay webhook exists for bookings** — confirmation is entirely client-driven. If the patient pays but closes the tab before `/payment-callback` runs, the Booking expires with money taken and no reconciliation. (The only Razorpay webhook in the repo is for doctor subscriptions, \$15.)

Cleanup: expired `pending_payment` rows are released opportunistically whenever slots are read, plus `GET /api/cleanup-reservations` exists (its `CRON_SECRET` check is commented out and it's not scheduled in `vercel.json`).

9. Priority Connect (LIVE, single-tenant)

- ✅ A separate ₹99/month “Priority Circle” product: direct WhatsApp access to Dr. Yuvaraj. Not an appointment — no slots, no calendar, no OTP.
- `/priority-connect` — marketing page with a **hardcoded** Razorpay Payment Button (`pL_SAxva2loHAV71c`). All copy hardcoded to Dr. Yuvaraj T.
- Razorpay redirects → `/priority-callback` → `POST /api/verify-priority-payment` : saves a payment record (inline `PriorityConnect` schema defined **inside the route**, not in `models/`), POSTs a Wylto webhook, then surfaces a WhatsApp deep link.
- ⚠️ Signature verification failures are logged but **never rejected**, and the callback UI shows success even on errors (deliberately optimistic).
- `POST /api/create-priority-order` exists (custom checkout path) but the page doesn’t call it — dead code.

10. Doctor Admin Dashboard

- ✅ LIVE at `/admin/dashboard` (client-guarded by `app/admin/dashboard/layout.js` → `/api/auth/me`).

Route	Feature
<code>/admin/dashboard</code>	Home: onboarding checklist, stats
<code>/admin/dashboard/pages</code> (+ <code>/new</code> , <code>/[id]</code> , <code>/homepage</code>)	Website Builder — create/edit BookingPages: add/reorder/toggle 18 section types, per-section config forms (<code>components/admin/booking-pages/config-forms/</code>), theme picker, image upload to Vercel Blob, publish/draft/archive, duplicate
<code>/admin/dashboard/bookings</code>	View/manage bookings (<code>GET/PATCH /api/doctor/bookings</code>)
<code>/admin/dashboard/slots</code>	Availability manager: TimeSlots, WeeklySchedule per mode/day, DateOverrides (block dates / custom slots)
<code>/admin/dashboard/modes</code>	Consultation modes — ⚠️ nav label says “ Clinic Manager ” but it’s modes CRUD, unrelated to the ClinicManager role

Route	Feature
/admin/dashboard/contacts	CRM (see §13)
/admin/dashboard/workflows	Drip campaigns (see §13)
/admin/dashboard/templates	Message templates
/admin/dashboard/ai-generate	AI website/content generation (Anthropic; requires <code>ANTHROPIC_API_KEY</code> — currently unset, “Coming Soon”)
/admin/dashboard/analytics	SlotView funnel analytics (Recharts)
/admin/dashboard/blog-articles (+ <code>/[id]</code>)	Blog editor (single-tenant BlogArticle model)
/admin/dashboard/forum	Forum Q&A moderation
/admin/dashboard/gmb/*	 GMB suite — hidden from nav by <code>SHOW_CONSULTATION_FEATURES = false</code> in the dashboard layout; routes still work if visited directly
/admin/dashboard/settings	Tabs: profile, contact, practice, domain (custom domain connect), subscription (Razorpay), seo (provision SEO users), clinic-manager (provision clinic managers)

Image uploads: `POST /api/admin/upload-image` → Vercel Blob (`BLOB_READ_WRITE_TOKEN`), public CDN URLs.

11. Platform Admin Dashboard

✅ LIVE at `admin.curago.in` (rewrites to `/platform-admin`). Single account from `PLATFORM_ADMIN_EMAIL` / `PLATFORM_ADMIN_PASSWORD` (plaintext env compare, no DB).

Route	Feature
/dashboard	Platform overview
/dashboard/doctors (+ <code>/new</code> , <code>/[id]</code> , <code>/[id]/website</code>)	List/create/edit doctors, suspend/activate, view any doctor’s site & bookings. ⚠️ Create-with-invite is half-built: invite token never persisted, welcome email is a <code>console.log</code> , generated password returned in the API response
/dashboard/bookings	All bookings across tenants
/dashboard/reputation-managers	CRUD reputation managers + assign doctors (checkbox multi-select)
/dashboard/analytics	Doctor growth, bookings by mode/time/weekday, top doctors

APIs: `/api/platform/*`, guarded by `requirePlatformAdmin` (`platform_admin_token`).

12. Sub-User Dashboards

All three reuse the mechanism described in §5: their tokens are accepted by `getCurrentDoctor()` as the owning doctor.

Clinic Manager (`/clinic-manager/dashboard`) — provisioned by the doctor (Settings → Clinic Manager tab → `POST /api/doctor/clinic-managers`). Sees **Contacts + Workflows** only; those pages literally re-export the doctor’s admin pages (`export { default } from '@app/admin/dashboard/contacts/page'`).

SEO User (`/seo/dashboard`) — provisioned by the doctor (Settings → seo tab → `POST /api/doctor/seo-users`). Sees **Website Builder + Blog Articles** (re-exported doctor pages). Despite the name, this is a content-editor role, not SEO tooling.

Reputation Manager (`/reputation-manager/dashboard`) — provisioned by the platform admin. Sees assigned doctors as cards → per-doctor **contacts** view (+ import). No review/GMB functionality currently exposed despite the name. Auth is an inline `jwt.verify ( getManager() )` in each `/api/reputation-manager/*` route.

13. Contacts CRM, Messaging & Workflows

✅ LIVE. Models: `Contact` , `ContactStatus` , `MessageTemplate` , `MessageLog` , `MessageQuota` , `Workflow` , `WorkflowExecution` . Engine: `lib/messaging.js` .

Contacts: manual add, Excel/CSV import (`POST /api/doctor/contacts/import`), export via ExcelJS (`GET /api/doctor/contacts/export`), custom status pipeline (6 defaults seeded per doctor), tags, per-contact `googleReviewLink` . Sources: manual / import / booking / website.

Direct messaging: `POST /api/doctor/messages/send` → `lib/messaging.js` : - Resolves template variables (`{{name}}` , `{{reviewLink}}` , `{{clinicName}}` , `{{doctorName}}`) - Checks + atomically deducts `MessageQuota` (100 SMS / 500 email per month, auto-resets) - SMS via Twilio (+91 E.164 normalization), email via Resend (HTML wrapper) - Every send logged to `MessageLog` (status + provider id)

Workflows (drip campaigns): - `Workflow.steps[]` : `{stepOrder, delayDays, delayHours, channel, templateId}` - 5 defaults seeded per doctor: Review Request, Quick Follow-up, Gentle Reminder, Email Only, SMS Blitz - Started per contact: `POST /api/doctor/workflows/start` → creates `WorkflowExecution` (`currentStepIndex` , `nextRunAt`) - **Advanced by cron** `GET /api/cron/workflow-processor` (daily 09:00): finds executions with `nextRunAt <= now` , checks `Subscription.isActive` + quota, sends the step, schedules the next one, logs per-step outcomes.

14. Google My Business / Reputation Suite (PAUSED)

🔒 Fully built, intentionally hidden (`SHOW_CONSULTATION_FEATURES = false` in `app/admin/dashboard/layout.js`). Library: `lib/gmb.js` .

OAuth connect flow: 1. `GET /api/doctor/gmb/connect` → Google auth URL (`access_type=offline` , `prompt=consent` , `state=doctorId`), scopes `business.manage` . 2. Google → `GET /api/doctor/gmb/callback` → token exchange, fetch accounts+locations. - One location → `GmbConnection` created immediately. - Multiple → stashed in an **in-memory Map** (`pendingConnections` , 10-min TTL — ⚠️ breaks on serverless/multi-instance) → `/admin/dashboard/gmb/select-locations` . 3. Tokens stored `select:false` ; auto-refresh via `getValidAccessToken()` ; status lifecycle active/expired/disconnected/error.

Capabilities (per-connection feature toggles on `GmbConnection`): - **Posts:** create/schedule GMB posts (`GmbPost`), published by `cron/gmb-publish` (⚠️ not scheduled in `vercel.json`), `retry ≤3` , analytics. - **Reviews:** sync (`GmbReview`), reply/delete-reply, AI-suggested replies (field exists, partially wired), stats aggregation. - **Q&A:** `GmbFaq` sync + answer. - **Insights:** `GmbInsight` — search keywords, impressions, clicks, direction requests.

Review-request interceptor funnel (the reputation product): 1. Doctor sends review requests (`ReviewRequest` + `ReviewRequestTemplate` , default delay 24h) — dispatched by `cron/review-requests` (daily 09:00) via **Wylto WhatsApp** . 2. Patient opens `/review/{trackingId}` → click tracked (`/api/review/{trackingId}/click`). 3. Patient rates (`/rating`): - **≥ 4 stars** → redirected to the real Google review page (`buildReviewLink(placeId)`) - **< 4 stars** → private feedback form (`/feedback`) — never reaches Google 4. Full funnel statuses on `ReviewRequest` : pending → sent → delivered → clicked → reviewed / intercepted / failed.

⚠️ Dead link: the GMB dashboard links to `/admin/dashboard/gmb/reviews` , which doesn't exist.

15. Subscriptions, Promo Codes & AI Tokens

Subscription (`lib/razorpaySubscription.js`): ₹1000/month Razorpay plan, 30-day trial auto-created at signup (`Subscription.getOrCreateTrial`), 120 cycles. - `POST /api/doctor/subscription/create` / `cancel` - `POST`

`/api/doctor/subscription/webhook` — **the only Razorpay webhook in the repo**; HMAC-verified with `RAZORPAY_WEBHOOK_SECRET` ; updates subscription status. - `Subscription.isActive` gates workflow messaging (checked in cron).

Promo codes (`PromoCode` , seeded via `scripts/seedPromoCode.js` , e.g. `CURAG050`): `POST /api/doctor/promo-code/redeem` → unlocks premium, credits 50 free SMS.

Reference codes (`ReferenceCode` , seeded via `scripts/seedReferenceCodes.js` , e.g. `CURAG02024` , `LAUNCH50`): optional gate at doctor signup. Overlaps with `PromoCode`.

AI tokens (`AIToken` wallet): `POST /api/doctor/ai-tokens/purchase` (Razorpay order) → `verify-payment` → balance credited; generate deducts. Backing `lib/aiGenerate.js` (Anthropic, model `claude-sonnet-4-20250514`) — currently disabled in production because `ANTHROPIC_API_KEY` is unset (“Coming Soon”).

16. Custom Domains

- ✔ LIVE. `lib/vercelDomains.js` + Doctor Settings → Domain tab.
1. Doctor enters domain → `POST /api/doctor/domain` → adds domain to the Vercel project via Vercel API (`VERCEL_API_TOKEN` / `VERCEL_PROJECT_ID`), stores on `Doctor.customDomain` .
 2. UI shows required DNS records (A / CNAME) from `getDomainConfig` .
 3. `POST /api/doctor/domain/verify` → Vercel verification.
 4. At request time, middleware resolves the domain via `GET /api/public/domain-lookup` (5-min in-memory cache) → rewrites to `/site/{subdomain}` .
- Plan doc: `docs/custom-domain-implementation-plan.md` .

17. Legacy Dr. Yuvaraj Site (LIVE on root domain)

✔ These root-domain pages are **live production surfaces** — hardcoded to Dr. Yuvaraj T (SRV Hospital Chembur, WhatsApp 917021227203 / 918369743571). Do not remove without a decision.

URL	What it is
<code>/about</code>	Doctor bio page (server component, hero/about/services/forum/CTA sections)
<code>/services</code>	GI/HPB surgery services (only legacy page with its own <code>metadata</code> export)
<code>/blog</code> , <code>/blog/[slug]</code>	Clinical blog — <code>BlogArticle</code> model, rigid 6-section template, “₹150 Surgical Audit” CTA → <code>/schedule-consultation</code> . Client components, no metadata
<code>/gbsi</code>	“Gut-Brain” (GBSI) marketing landing
<code>/digital-clinic</code>	Telemedicine landing with Razorpay + application modal
<code>/myclinic</code> , <code>/myclinic/[slug]</code>	Clinic listing → renders published <code>BookingPages</code> via <code>SectionRenderer</code> (supports <code>?preview</code> with <code>adminToken</code>)
<code>/priority-connect</code> , <code>/priority-callback</code>	Priority Circle ₹99 product (\$9)
<code>/schedule-consultation</code>	👤 Old “request appointment” form — no slot check, no OTP; POSTs to a Wylto webhook and a hardcoded Google Apps Script URL
<code>/schedule-confirmation</code> , <code>/payment-callback</code>	Booking confirmation/callback pages (also used by dormant payment flow)

URL	What it is
<code>app/page.old.js</code>	👤 Unrouted old homepage (root <code>/</code> is served by <code>app/(marketing)/page.js</code>)

Google Apps Script era: `complete-google-apps-script.js` (89 KB) and `slot-booking-google-script.js` at repo root are the source of the old Google Sheets-based booking/payment/invoice system (pasted into script.google.com, not part of the Next.js build). Only `/schedule-consultation` still pings that deployment.

Other legacy artifacts: `lib/slotManager.js` + `data/*.json` (file-based slot store, writes to `/tmp` on Vercel — ephemeral), `models/Slot.js`, `POST /api/book-consultation` (uses the file store, no known caller), `lib/db.js` (duplicate of `lib/mongodb.js`).

18. Analytics & Tracking

Three independent layers:

- 1. Server-side booking funnel** — `SlotView`: every “View Slots” click stores patient details + page/referrer/UA via public `POST /api/track-slot-view`. Doctor dashboard `/admin/dashboard/analytics` reads `GET /api/admin/analytics/slot-views` (total views, unique users, conversion rate, breakdowns by page/date/gender/referrer; capped 1000 records). ⚠️ `convertedToBooking` / `bookingId` linking is defined but never populated. Docs: `ANALYTICS_IMPLEMENTATION.md`; health check: `test-analytics.js`.
- 2. Client-side GTM/GA4** — GTM container `GTM-PL6KV3ND` loaded lazily in `app/layout.js` from a **custom server-side GTM host** (`gtm.curago.in` / `gtm.curago.co.in`) + noscript iframe. `lib/tracking.js` pushes `dataLayer` events (`page_view`, `purchase`, `appointment_booked`, `whatsapp_click`, `form_submit`, `scroll depth`, `ref` referral capture). ⚠️ purchase amounts are hardcoded (150 / 99).
- 3. Platform analytics** — `/api/platform/analytics/overview` (doctor growth, booking distributions). GMB insights are separate (§14).

19. Cron Jobs

Registered in `vercel.json` (both daily 09:00 UTC; all cron routes expect `Authorization: Bearer $CRON_SECRET`):

Endpoint	Job
<code>/api/cron/workflow-processor</code>	Advances due <code>WorkflowExecution</code> steps (subscription + quota gated)
<code>/api/cron/review-requests</code>	Sends due <code>ReviewRequest</code> s via Wylto WhatsApp (retry ≤ 3)

Exist but NOT scheduled (must be hit externally or are orphaned): - `/api/cron/gmb-publish` — publishes scheduled GMB posts (paused feature anyway) - `/api/cleanup-reservations` — expires stale `pending_payment` bookings (⚠️ its auth check is commented out; cleanup otherwise happens opportunistically on slot reads)

20. SEO — Current State

Mostly **absent** — the biggest structural gap for a healthcare product:

- ❌ No `sitemap.xml` / `sitemap.js`, no `robots.txt` / `robots.js` anywhere.
- ❌ No JSON-LD / schema.org structured data (no `Physician`, `MedicalBusiness`, `Article`, `FAQPage` markup) despite ideal content for it.
- ❌ Blog (`/blog`, `/blog/[slug]`), `/digital-clinic`, `/myclinic`, `/priority-connect` are client components with **no metadata exports**.
- ❌ `app/layout.js` references `/og-preview.jpg` for OG/Twitter — the file doesn’t exist in `public/` → broken social previews site-wide.
- ❌ Doctor sites: `generateMetadata` reads `bookingPage.ogImage`, a field the schema doesn’t have → no OG image ever.

-  Doctor subdomains have no subpages at all (only homepage), so no per-doctor content/SEO surface area.
-  What exists: root `metadata` in `app/layout.js` (title template, keywords, robots index/follow, `metadataBase`), marketing canonical `https://curago.in` , `/services` static metadata, doctor-site title/description from `BookingPage`.
- The “SEO” dashboard/role (§12) is a content-editor login, not SEO tooling.

21. Scripts & Maintenance Tools

Standalone Node scripts in `scripts/` , run manually, load `.env.local` :

Script	Purpose
<code>seedPromoCode.js</code>	Seed master promo <code>CURAG050</code> (unlimited)
<code>seedReferenceCodes.js</code>	Seed signup codes <code>CURAG02024</code> , <code>LAUNCH50</code> , <code>PARTNER01</code> , <code>DEMO</code>
<code>migrate-to-new-db.js</code>	Copy legacy (no-doctorId) data between DBs (<code>SOURCE_DB_URI</code> / <code>DEST_DB_URI</code>) —  <code>console.logs</code> a live connection string with credentials
<code>migrate-clinics.js</code>	Backfill display fields on legacy booking pages (hardcoded slugs)
<code>fix-all-indexes.mjs</code>	Drop legacy global-unique indexes that break multi-tenancy
<code>fix-slug-index.js</code> , <code>fix-consultation-mode-index.js</code> , <code>fix-timeslot-index.js</code>	Targeted index repairs
<code>fix-orphan-modes.js</code>	Repair consultation modes with no doctor
<code>check-subdomain.js</code> , <code>check-all-subdomains.js</code> , <code>check-consultation-modes.js</code>	Read-only diagnostics

Root-level setup docs: `BOOKING_SETUP.md` , `GOOGLE_MEET_SETUP.md` , `DOMAIN_WIDE_DELEGATION_SETUP.md` , `DEPLOYMENT.md` , `ANALYTICS_IMPLEMENTATION.md` , `CURRENT_SYSTEM_CAPABILITIES.md` , `IMAGE_UPLOAD_DEBUG.md` , `FIX_INSTRUCTIONS.md` .

22. Complete API Route Reference

Auth legend: **D** = doctor JWT (`requireDoctorAuth` / `getCurrentDoctor` ; also accepts seo/clinic-manager tokens), **L** = legacy `isAuthenticated` , **P** = platform admin, **C** = `Bearer CRON_SECRET` , `-` = public.

Doctor auth (`/api/auth/*`)

Route	Methods	Auth	Purpose
<code>/api/auth/signup</code>	POST	-	Register doctor (+trial, email OTP)
<code>/api/auth/login</code> / <code>logout</code>	POST	-	Session cookie in/out
<code>/api/auth/me</code>	GET	D	Current doctor
<code>/api/auth/verify-email</code>	POST/PUT	-	Email OTP verify/resend
<code>/api/auth/forgot-password</code> / <code>reset-password</code>	POST(/GET)	-	Password reset
<code>/api/auth/check-subdomain</code>	GET/POST	-	Subdomain availability

Doctor portal (/api/doctor/*) — all D

profile, settings, booking-pages (+[id]), bookings, modes, slots, schedule, clinics (+[id]), meetings, contacts (+[id], export, import, statuses), templates (+[id]), workflows (+[id], start, executions), messages/quota, messages/send, clinic-managers, seo-users, ai-tokens (+generate, purchase, verify-payment), subscription (+create, cancel, **webhook** ← public, signature-verified), promo-code/redeem, domain (+verify), gmb (+status, connect, **callback** ← public OAuth redirect, disconnect, select-locations, posts +[id], requests, templates)

Admin (legacy-mixed, /api/admin/*)

Route	Auth	Purpose
/api/admin/login	–	 env-credential login (Bearer in body)
booking-pages (+[id], duplicate)	D	Page builder CRUD
blog-articles (+[id])	D/L	Blog CRUD
consultation-modes , slots , time-slots (+reset), weekly-schedule , date-overrides , available-times	D/L	Scheduling admin
forum (+[id])	D/L	Forum moderation
analytics/slot-views	D/L	Funnel analytics
upload-image , cleanup-images , migrate-clinics	L only	Blob uploads / maintenance

Public booking & content

Route	Methods	Purpose
/api/consultation-modes	GET	Doctor's active modes (by doctorId/subdomain)
/api/available-slots	GET	Effective slots for date+mode
/api/send-otp	POST	✅ OTP via Wylto + Twilio
/api/verify-otp-and-book	POST	✅ Verify OTP → calendar → confirm booking
/api/reserve-slot	POST	 10-min payment hold
/api/verify-payment	POST	 Razorpay verify → confirm
/api/track-slot-view	POST	Funnel analytics write
/api/book-consultation	POST	 file-store booking (no caller)
/api/schedule-consultation	POST	 old form → Wylto + Apps Script
/api/create-priority-order , /api/verify-priority-payment , /api/priority-circle	POST	Priority Circle
/api/booking-pages/[slug]	GET	Public page JSON (?preview w/ token)
/api/blog-articles (+[slug])	GET	Published blog (global, no tenant filter)
/api/forum	GET/POST	Public Q&A
/api/clinics	GET	Published clinic pages by category
/api/review/[trackingId]/click / rating / feedback	POST	Review interceptor
/api/public/[subdomain]/site	GET	Doctor site JSON

Route	Methods	Purpose
/api/public/domain-lookup	GET	Custom domain → subdomain (middleware)
/api/cleanup-reservations	GET	Expire stale holds (auth commented out)
/api/debug/check-subdomain	GET	Debug helper

Platform (/api/platform/*) — all P except login/logout

auth/login, auth/logout, auth/me; doctors (+create, [id], [id]/suspend, [id]/bookings); bookings; analytics/overview; reputation-managers (+[id])

Sub-user auth (inline jwt.verify per route)

- /api/clinic-manager/auth/login + me (cookie clinic_manager_token)
- /api/seo/auth/login + me (cookie seo_token)
- /api/reputation-manager/auth/login + me ; doctors , doctors/[id]/contacts (+import) (cookie rep_manager_token)

Cron — all C

/api/cron/workflow-processor , /api/cron/review-requests (scheduled); /api/cron/gmb-publish (unscheduled)

23. Known Issues & Technical Debt

Security

1. **Sub-role privilege escalation by design:** getCurrentDoctor() accepts seo_token / clinic_manager_token and returns full doctor identity — SEO and clinic-manager users can call **every** /api/doctor/* and /api/admin/* endpoint (bookings, settings, domain, subscription...), not just what their UI shows.
2. **Single JWT secret for all roles** — tokens differ only by role claim; doctor and legacy admin even share the cookie name doctor_token .
3. **Platform admin = one env credential, plaintext compare, no rate limiting.**
4. **Client-side-only dashboard guards** — no server middleware protection on any dashboard route.
5. /api/cleanup-reservations auth check commented out (open endpoint).
6. scripts/migrate-to-new-db.js logs a live MongoDB connection string with credentials.
7. verify-priority-payment never rejects — any payment_id records a “completed” ₹99 payment.
8. Platform doctor-create returns generated passwords in the API response / console.log.

Reliability

9. **No Razorpay webhook for bookings** (payment flow) — client-driven confirmation can lose paid bookings. (Dormant path, but a landmine if payment mode is enabled.)
10. **Calendar creation is a hard dependency** in verify-otp-and-book and verify-payment — a Google API failure blocks an already-OTP-verified (or paid) booking.
11. send-otp falls back to the **first active doctor** when subdomain can't be resolved — can misattribute bookings (guaranteed on localhost).
12. GMB OAuth pendingConnections in-memory Map breaks on serverless (multi-location connects will randomly fail).
13. cleanup-reservations and gmb-publish not in vercel.json crons.

SEO (see §20)

14. No sitemap, no robots.txt, no JSON-LD; blog has no metadata; og-preview.jpg missing; bookingPage.ogImage not in schema; doctor subdomains have no subpages.

Code health / duplication

15. `lib/db.js` $\equiv$ `lib/mongodb.js` (duplicate connectors); `lib/slotManager.js` + `models/Slot.js` + `data/*.json` legacy file store; `PromoCode` vs `ReferenceCode` overlap; two section renderers (inline switch vs `SectionRenderer.js`) that can drift; `getSubdomainFromRequest` copy-pasted in ≥ 3 routes; calendar+webhook+email+SMS block duplicated between `verify-otp-and-book` and `verify-payment`.
 16. Un-tenanted models bleed across tenants: `BlogArticle`, `ForumPost`, `SlotView`.
 17. Misleading nav: "Clinic Manager" menu item opens consultation **modes**.
 18. Dead links/routes: `/privacy`, `/terms` (marketing footer), `/admin/dashboard/gmb/reviews`, `/doctor/complete-profile` (invite email), `app/page.old.js` unrouted.
 19. Hardcoded values: purchase tracking amounts (150/99), Razorpay button ids, Dr. Yuvaraj contact details across legacy pages, default `razorpayButtonId` in the `BookingPage` schema.
 20. `SlotView.convertedToBooking` never set — funnel conversion metric is not real.
-

Generated from a full-codebase review on 2026-07-09 (branch `main`, commit `aa880ea1`).